

Benchmarking Speculative Decoding for Apertus LLM Serving

Project proposal · Swiss AI Initiative

STUDENT	COURSE	SCOPE
Faruk Zahiragic	COM-508 · Optional Research Project in Data Science	8 ECTS · 14 weeks · Autumn 2026

■ Abstract

Large language model inference is expensive because standard autoregressive decoding invokes the full model once for every output token. Speculative decoding attempts to reduce this sequential bottleneck: a smaller model proposes several tokens, and the target model verifies them together while preserving the target model's output distribution. Whether this is beneficial in practice depends on acceptance rate, workload, batching, parallel layout, and the draft model's compute and memory overhead.

This project will determine whether an 8B draft model should be used to accelerate production serving of **Apertus-v1.5-70B**. It will build a reproducible evaluation on one four-GPU GH200 Clariden node, compare ordinary decoding, 8B-draft speculation, and model-free n-gram speculation, and measure both performance gains and operational costs. The outcome will be an evidence-based recommendation identifying where speculation helps, where it hurts, and which settings should be deployed.

■ Motivation and research gap

Apertus is an openly released multilingual LLM developed within the Swiss AI Initiative. Efficient serving is essential for making the 70B model broadly usable, but speculative-decoding results from other models do not transfer directly: speedup depends on the target-draft agreement, GPU kernels, tensor parallelism, context length, and server load. No controlled characterization currently establishes this trade-off for the Apertus 8B/70B pair on the Swiss AI serving stack.

The project extends prior Swiss AI performance-benchmarking practice from architectural serving studies to an inference algorithm whose benefit is explicitly workload-dependent. It treats a negative result as scientifically useful: the aim is not to demonstrate a speedup at any cost, but to locate the production break-even conditions.

CORE OBJECTIVE

Quantify the latency, throughput, capacity, and memory effects of speculative decoding for Apertus-v1.5-70B, explain those effects through acceptance and systems measurements, and recommend a robust production configuration—or recommend keeping ordinary decoding.

■ Research questions

1. How do workload type and request concurrency affect speculative acceptance, time to first token (TTFT), time per output token (TPOT), and throughput?
2. Which speculative depth and draft tensor-parallel layout provides the best performance envelope, and when does draft overhead outweigh accepted work?
3. When is model-free n-gram speculation preferable to maintaining an 8B draft model?
4. Does the best latency configuration improve sustainable request rate without an unacceptable tail-latency or KV-cache-capacity penalty?

■ Experimental methodology

System and controlled factors

Experiments will run with vLLM on one Clariden GH200 node (four GPUs), using target tensor parallelism of four and cached model weights. The software revisions, launch configuration, node identity, prompt corpus, request order, generation parameters, and output limits will be recorded. Greedy decoding is the core experiment.

Serving configurations

- **Baseline:** Apertus-v1.5-70B without speculation.
- **Draft model:** Apertus-v1.5-8B with speculative depth $k \in \{2, 3, 5, 8\}$ and draft tensor parallelism $\in \{1, 4\}$.
- **Model-free baseline:** vLLM n-gram speculation at the same depths.

Workloads and load levels

A deterministic, versioned prompt set will cover:

- open-ended chat;
- code generation;
- long-context summarisation (16k-64k input tokens).

Closed-loop tests will use concurrency 1, 8, and 32. A final open-loop experiment will estimate sustainable request rate under fixed latency and error-rate gates.

User latency	Throughput	Mechanism	Resources
TTFT and TPOT, reported at p50 and p95	Output tokens/s and sustainable requests/s	Acceptance rate, accepted length, per-position acceptance	GPU memory, KV-cache capacity, utilisation and preemptions

Validity and analysis

First, exact greedy outputs will be compared on a smoke corpus as an implementation sanity check. The full study will use identical prompts and sampling settings across variants, warm-ups, immutable raw results, and independently launched deployment repeats. Screening uses two repeats; selected configurations are confirmed with five repeats at all concurrency levels. Variant order will be counterbalanced, and interleaved baselines will detect runtime drift. Results will include absolute metrics, speedup ratios, deployment-level spread, and 95% bootstrap confidence intervals. Runs with server restarts, counter resets, or below 99% request success will be rejected and documented.

DECISION RULE

Recommend a setting only if confirmation runs show a repeatable gain on at least one declared workload, no material regression on the others, and no unacceptable tail-latency or memory-capacity penalty. A workload-specific recommendation is permitted only when production traffic can be routed accordingly.

■ Schedule and effort

Period	Milestone
Preparation · 30 h	Finalize scope with supervisors; focused literature review; cluster and benchmark-harness preparation.
Weeks 1-2	Reproduce baseline and draft serving; validate metrics; greedy correctness checks; submit detailed schedule.
Weeks 3-4	Curate and tokenize workload strata; calibrate the load generator; freeze the experimental protocol.
Weeks 5-7	Screen draft depth/parallelism and n-gram settings at concurrency 1 and 8.
Weeks 8-10	Confirm selected configurations at concurrency 1, 8, and 32 with independent deployment repeats.
Weeks 11-12	Capacity and KV-cache study; targeted GPU profiling if needed; statistical analysis.
Weeks 13-14	Synthesis, reproducibility audit, recommendation, report draft, and presentation preparation.
Finalisation · 30 h	Complete and submit the technical report; prepare and deliver the laboratory presentation.

Planned total: approximately 228 hours, aligned with the IC workload model (30 h preparation + 14 weeks at about 12 h/week + 30 h final reporting/presentation).

■ Expected contributions

- The first controlled performance characterization of draft-model and n-gram speculative decoding for Apertus-v1.5-70B on GH200 hardware.
- A mechanistic account connecting workload, acceptance, speculative depth, parallel layout, batching, and memory cost to observed speedup.
- A reproducible benchmark protocol and reusable tooling for future Apertus releases.
- A production recommendation with explicit winning and losing regimes, rather than a single decontextualized speedup number.

■ Deliverables

1. Version-controlled launch, workload, measurement, and analysis code in the `swiss-ai/apertus-speculative-decoding` repository.
2. Raw request-level measurements, metric snapshots, launch provenance, and analysis-ready summaries.
3. A concise scientific report containing the literature review, methods, results, limitations, and deployment recommendation.
4. An oral presentation organized by the hosting laboratory.
5. If results support deployment, a pull request proposing tuned defaults for the Swiss AI model-launch configuration.

■ Feasibility and risk management

The target and draft models share a tokenizer, an existing vLLM speculative configuration is available, and a benchmark scaffold has already been prepared. The core matrix fits one four-GPU node and is staged so weak configurations can be eliminated before expensive repeats.

Risk	Mitigation
Limited cluster availability	Short staged jobs, cached weights, counterbalanced runs, and prompt node-hour release.
Runtime or metric instability	Pinned software, raw logs/counters, smoke tests, and explicit rejection criteria.
Matrix too large	Predeclared screening before confirmatory repeats; stretch goals remain optional.
Network measurement noise	On-cluster load generation and separate reporting of gateway effects.

Optional extensions: the Apertus “thinking” variant, temperature 0.8, and deeper kernel profiling will be attempted only after the core conclusion is stable.

■ Supervision, assessment, and compliance

The project is designed as individual and independent research under the oversight of an EPFL academic supervisor and the hosting EPFL laboratory, with a clearly identified day-to-day technical supervisor. Progress will be reviewed in weekly meetings; a detailed schedule will be agreed by the end of week 2. Assessment evidence will cover work and initiative during the semester, reproducible code and results, a scientific report, and an oral presentation. The project will be registered in IS-Academia as COM-508, conducted during the 14-week semester, and not undertaken concurrently with the 12-ECTS mandatory research project. It is an ECTS-credited, non-remunerated academic project.

■ Data, reproducibility, and responsible resource use

Experiments will use public or appropriately licensed text/code sources and will not require personal or sensitive data. Prompt IDs, tokenizer-length distributions, software revisions, configuration files, node/job provenance, and analysis procedures will be preserved. The staged design, fixed validity gates, and release of negative as well as positive findings reduce selection bias and unnecessary GPU use.

Selected references

1. Leviathan, Y., Kalman, M., and Matias, Y. “Fast Inference from Transformers via Speculative Decoding.” ICML, 2023.
2. Chen, C. et al. “Accelerating Large Language Model Decoding with Speculative Sampling.” 2023.
3. Kwon, W. et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention.” SOSP, 2023.
4. EPFL IC. “Master Research Project” and “IC Quick Guide to Research Projects.” Requirements consulted 31 August 2026.
5. EPFL Coursebook. “COM-508 — Optional Research Project in Data Science.” 2026–2027 edition.